Attention Residual as a Parameter-Efficient Fine-Tuning Method

Shaowei Zhang, Weiguo Li

March 31, 2026

Open Source & Reproducibility

https://github.com/shaoweizhang1/AttnRes_PEFT
https://wandb.ai/zsw441296721/AttnRes_PEFT

Abstract

We study AttnRes-PEFT, a parameter-efficient fine-tuning method that adapts the Attention Residuals (AttnRes)[6] idea to downstream tasks. Instead of updating backbone weight matrices, AttnRes-PEFT adds a lightweight depth-wise adapter at each Transformer layer. The adapter attends over earlier hidden states and adds a gated residual correction to the frozen backbone output. We evaluate the method on BoolQ, GSM8K, and RTE, and compare it with a frozen base model and LoRA. AttnRes-PEFT substantially improves over the frozen model on GSM8K and remains competitive on BoolQ, but LoRA is stronger overall on the three tasks in our setting. We also conduct a lookback ablation on RTE and find that performance saturates quickly once the model can attend to a small number of previous layers. Finally, we analyze layer usage patterns to study which earlier layers are most frequently reused and how these patterns differ across tasks.

Keywords: parameter-efficient fine-tuning; attention residuals; LoRA; benchmarking; NLP evaluation

Author roles: Shaowei Zhang: Software, Data curation, Validation, Investigation, Experiment design, Writing – review & editing. Weiguo Li: Conceptualization, Methodology, Software, Analysis, Writing – original draft.

1 Introduction

Large pre-trained language models such as GPT-5 and Claude have demonstrated remarkable capabilities across a wide spectrum of natural language understanding and generation tasks. However, full fine-tuning of these models is increasingly prohibitive: a model with billions of parameters requires storing a complete set of gradients, optimizer states, and updated weights for every downstream task, quickly exhausting both GPU memory and storage budgets. Parameter-efficient fine-tuning (PEFT) addresses this challenge by updating only a small fraction of parameters while freezing the pre-trained backbone. The dominant PEFT paradigm, LORA[4], inserts low-rank weight decompositions that learn task-specific weight deltas. Bottleneck adapters [3] insert small feed-forward modules between Transformer sub-layers. Prefix tuning [5] prepends learnable tokens to the key-value sequence. All of these methods share a common characteristic: they operate within a single layer at a time, leaving the depth-wise information flow—the way representations evolve across layers—completely unchanged.

Meanwhile, a parallel line of research has questioned whether standard residual connections, which uniformly accumulate all layer outputs with unit weights, are an optimal architectural choice for depth. Chen et al. (2026)[6] introduced **Attention Residuals (AttnRes)** as a replacement for fixed-weight residual accumulation during pre-training. Instead of adding only the immediately preceding layer output to the current representation, AttnRes uses a learned, input-dependent softmax over all preceding layer outputs, allowing each layer to selectively borrow the most useful representation from any depth.

In this paper, we ask: can the core idea of Attention Residuals be repackaged as a PEFT method for downstream fine-tuning? The answer is yes, and the resulting method—**AttnRes-PEFT**—occupies a fundamentally different region in the PEFT design space. Rather than changing how individual layers transform their input, AttnRes-PEFT changes how representations are aggregated across depth, enabling the adapter to perform lightweight but expressive depth-wise routing of information without touching any pre-trained weight matrices.

Our contributions are:

- We design AttnRes-PEFT, a PEFT adapter that applies depth-wise attention aggregation over frozen backbone hidden states.
- We show that zero-gate initialization makes the adapter contribution exactly zero at initialization, ensuring safe and stable training.

- We introduce a lookback-window variant and analyze its parameter count relative to LoRA.
- We position AttnRes-PEFT in the broader PEFT taxonomy and discuss its unique design tradeoffs.

2 Background

2.1 Transformer Residual Connections and PreNorm

A standard Transformer layer with PreNorm computes:

$$h^{(\ell)} = h^{(\ell-1)} + F^{(\ell)}\left(\text{LayerNorm}\left(h^{(\ell-1)}\right)\right)$$

where F is a sub-layer (attention or feed-forward). The residual stream h is a running sum of all sub-layer outputs from layer 0 to layer ℓ. This uniform accumulation assigns equal, unit weight to every depth, which AttnRes[6] observe causes progressive dilution of individual layer contributions as depth grows. The norm of h grows with depth, reducing the relative influence of each newly added term.

2.2 Attention Residuals (AttnRes)

AttnRes [6] replaces the fixed residual accumulation with a learned, content-dependent aggregation. At each layer ℓ and token position t, the new hidden state is:

$$h_t^{(\ell)} = \sum_{k \le \ell} \alpha_{k,t}^{(\ell)} h_t^{(k)} \tag{1}$$

where α is a softmax over preceding depths, computed by attending the current representation to earlier layer outputs. This allows each layer to dynamically route through depth, selecting the most relevant past representation rather than blindly summing them all. The Kimi team demonstrate that AttnRes mitigates the PreNorm dilution problem and leads to more uniform output magnitudes and gradient distributions across depth.

2.3 Parameter-Efficient Fine-Tuning

PEFT methods adapt a frozen pre-trained model by training a small number of additional parameters. We briefly summarize the most relevant approaches:

LoRA Low-rank adaptation [4] decomposes the weight update ΔW as $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ are low-rank matrices ($r \ll d$). Only A and B are trained. LoRA introduces $2dr$ parameters per weight matrix.

Bottleneck Adapters [3] insert a down-projection followed by a non-linearity and up-projection between Transformer sub-layers. The bottleneck dimension r controls the parameter budget, yielding $2dr$ parameters per adapter.

Prefix Tuning [5] prepends L learnable virtual tokens to the key and value sequences of each layer, introducing $2Ld$ additional parameters but leaving weight matrices untouched.

All three methods operate on the weight matrices of individual layers. None of them modify the depth-wise aggregation of representations across layers—the dimension that AttnRes-PEFT directly targets.

3 AttnRes-PEFT

3.1 Overview

AttnRes-PEFT wraps a frozen pre-trained Transformer (in our case, Qwen series) and attaches a lightweight adapter to each layer. The adapter intercepts the frozen layer's output and computes a depth-aware residual correction that is added before passing to the next layer. Formally, let $h_{\text{base}}^{(\ell)}$ denote the output of the frozen backbone layer ℓ. The adapter collects the sequence of all preceding hidden states $\{h^{(0)}, \ldots, h^{(\ell-1)}\}$:

$$h_{\text{out}}^{(\ell)} = h_{\text{base}}^{(\ell)} + g^{(\ell)} \cdot \Delta h_t^{(\ell)} \tag{2}$$

where $g^{(\ell)}$ is a learnable scalar gate and $\Delta h_t^{(\ell)}$ is the depth-weighted residual correction, described below.

3.2 Depth-Wise Attention Aggregation

At layer ℓ and token position t, the adapter computes attention weights over all preceding layer outputs. In the current implementation, each layer uses a learned query vector $q^{(\ell)} \in \mathbb{R}^d$ (not a projection matrix). Source states are first normalized:

$$\tilde{h}_t^{(k)} = \text{RMSNorm}_{\text{score}}^{(\ell)}\left(h_t^{(k)}\right), \quad k < \ell \tag{3}$$

and attention logits are computed by dot product with the query vector:

$$e_{k,t}^{(\ell)} = \left\langle q^{(\ell)}, \tilde{h}_t^{(k)} \right\rangle \tag{4}$$

Depth-wise attention weights are obtained by softmax over the depth axis:

$$\alpha_{k,t}^{(\ell)} = \frac{\exp(e_{k,t}^{(\ell)})}{\sum_{k'<\ell} \exp(e_{k',t}^{(\ell)})} \tag{5}$$

The residual correction is then:

$$\Delta h_t^{(\ell)} = \mathrm{RMSNorm}_{\mathrm{out}}^{(\ell)} \left(\sum_{k<\ell} \alpha_{k,t}^{(\ell)} \cdot h_t^{(k)} \right) \tag{6}$$

This formulation is inspired by the AttnRes mechanism proposed by the Kimi Team [6], and adapts its depth-wise routing idea with a simplified scorer for PEFT. In our implementation only $q^{(\ell)}$, the two RMSNorm scale vectors, and the gate $g^{(\ell)}$ are trainable. All backbone weights and all source hidden states remain frozen.

3.3 Scalar Gate and Safe Initialization

A key design choice is initializing the scalar gate $g^{(\ell)}$ to zero for all layers. At initialization, the adapter contribution is:

$$g^{(\ell)} \cdot \Delta h_t^{(\ell)} \Big|_{\mathrm{init}} = 0 \tag{7}$$

This means the model at the start of fine-tuning behaves exactly as the frozen backbone, regardless of the values of the query parameters. This property—which we call zero-correction initialization—is essential for stability. It allows the optimizer to discover useful depth-wise routing signals gradually, without disrupting the pre-trained representations from the first gradient step. As training progresses, the gate is free to grow, allowing the adapter to exert progressively stronger influence. This is analogous to the zero-initialization used in LoRA's matrix B [4], which ensures $\Delta W = BA = 0$ at initialization.

3.4 Lookback Window Variant

Full AttnRes-PEFT attends over all ℓ preceding layers, which means layer L (the last layer) must cache representations from all L preceding layers simultaneously. This storage grows linearly with model depth and can become significant for very deep models. Motivated by Block AttnRes[6] in the original paper, we introduce a lookback window variant that restricts the depth attention to the W most recent preceding layers:

$$\Delta h_t^{(\ell)} = \sum_{k=\max(0,\ell-W)}^{\ell-1} \alpha_{k,t}^{(\ell)} \cdot h_t^{(k)} \tag{8}$$

This reduces the number of cached states from $O(L)$ to $O(W)$ at inference and fine-tuning time, while preserving local depth-routing. In practice, a window of $W = 4$–8 retains most of the representational benefit of full attention residuals, as nearby layers tend to carry the most relevant intermediate representations.

3.5 Trainable Parameter Count

For a model with L Transformer layers and hidden dimension d, the total number of trainable parameters in AttnRes-PEFT is:

$$\Theta_{\mathrm{AttnRes}} = L \cdot (3d + 1) \tag{9}$$

In the current implementation, each layer contains a learned query vector of size d, two RMSNorm scale vectors of size d, and one scalar gate. Therefore, the per-layer trainable parameter count is $d + d + d + 1 = 3d + 1$.

For comparison, LoRA applied to the query, key, value, and output projections introduces $8Ldr$ trainable parameters, since each projection contributes $2dr$ parameters and four projections are adapted per layer. This makes AttnRes-PEFT substantially more parameter-efficient in terms of trainable weights, although its computation and memory cost still grow with the lookback window. Table 1 summarizes the comparison.

Table 1: Comparison of PEFT methods per Transformer layer (d = hidden dim, r = rank/bottleneck, L = number of layers).

Method	**Params (per layer)**	**Cross-depth**	**Weight-matrix change**
LoRA (Q,K,V,O)	$8dr$	No	Yes ($\Delta W = BA$)
Bottleneck Adapter	$2dr + r$	No	No
Prefix Tuning	$2Ld$ (total)	Partial (KV)	No
AttnRes-PEFT (ours)	$\mathbf{3d + 1}$	Yes (full depth)	**No**

4 Experimental Setup and Data Resources

4.1 Data Curation and Tasks

To ensure a controlled comparison between PEFT methods, we curated a suite of standard NLP benchmarks covering diverse reasoning and understanding capabilities. The selected tasks include:

- **GSM8K** [2]: A dataset of grade school math word problems requiring multi-step reasoning.
- **RTE** [7]: A textual entailment task from the GLUE benchmark.
- **BoolQ** [1]: A boolean question answering task requiring reading comprehension.

All raw datasets were preprocessed into a unified JSON format to standardize the input pipeline. Each sample contains the fields `instruction`, `input`, `output`, `task`, and `split`. No additional sampling strategies were employed beyond the original train/validation/test splits provided by the respective benchmarks to maintain ecological validity. Quality control measures included verifying prompt consistency across tasks and ensuring that training and evaluation structures remained identical for all methods.

4.2 Implementation Pipeline

The experimental resource was constructed through a four-stage pipeline:

1. **Data Standardization:** Conversion of raw task data into the shared JSON schema.
2. **Base Model Initialization:** Loading a frozen causal language model ("Qwen/Qwen2.5-7B") as the backbone.
3. **Adapter Integration:** Injecting trainable parameters via two distinct methods:
   - **LoRA** [4]: Standard low-rank updates applied to attention projection layers.
   - **AttnRes-PEFT**: The proposed method (see Section 3), where each Transformer layer receives a lightweight adapter that computes a depth-aware residual correction based on preceding hidden states.
4. **Execution:** Running training and evaluation scripts to generate checkpoints, predictions, and performance metrics.

Technical safeguards were implemented to ensure experimental integrity. Specifically, AttnRes checkpoints were configured to save only adapter parameters (excluding the frozen backbone), and decoder-only generation utilized left-padding during evaluation to prevent position embedding shifts. Hyperparameters were matched across methods where applicable to isolate the effect of the adaptation mechanism.

4.3 Hyperparameter Settings

We used matched optimization settings for LoRA and AttnRes-PEFT in the main experiments: 3 training epochs, learning rate 2×10^{-4}, gradient accumulation 8, and `save_strategy=epoch`. We set `fp16=True` when CUDA was available.

For LoRA, we used the same configuration across tasks: `r=8`, `alpha=16`, `dropout=0.05`, and target modules {`q_proj, k_proj, v_proj, o_proj`}. For AttnRes-PEFT in the main comparison, we used `lookback=8` and `gate_init=0.0` for all tasks. In the RTE lookback ablation, we evaluated `lookback` $\in \{1, 2, 4, 8, 16, 20, 28\}$.

Table 2: Task-specific sequence length and batch settings used in training and evaluation.

Task	Train max_length	Train batch	Eval max_length	Eval max_new_tokens
BoolQ	512	1	512	8
GSM8K	1024	2	1024	64
RTE	256	2	256	8

Limitations The scope of the current resource is limited to three English-language benchmarks and a single model family (Qwen-2.5). Generalization to other model architectures, multilingual tasks, or diverse hardware settings requires further validation. Users extending this work should verify licensing constraints associated with the original source datasets before public redistribution.

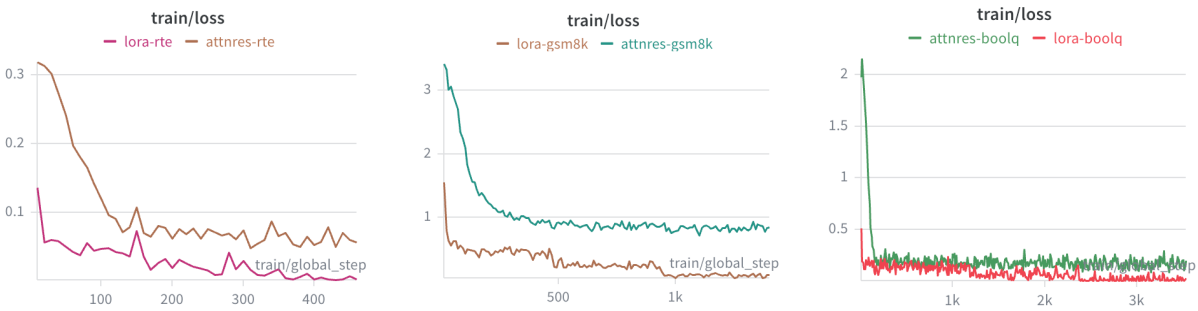
5 Experiments

5.1 Main Results

We compare **AttnRes-PEFT** against two baselines: (i) the frozen backbone without adaptation (*Base*) and (ii) **LoRA** [4] as a strong parameter-efficient fine-tuning baseline. Evaluation is conducted on three benchmarks: BoolQ, GSM8K, and RTE.

Table 3: Main results across three benchmarks. Accuracy (%) is reported for all tasks.

Method	BoolQ	GSM8K	RTE
Base (Frozen)	74.28	1.59	63.54
LoRA	**76.88**	**20.62**	**66.79**
AttnRes-PEFT (ours)	74.89	19.11	63.18



(a) RTE (b) GSM8K (c) BoolQ

Figure 1: Training loss comparison between LoRA and AttnRes-PEFT across three benchmarks: (a) RTE, (b) GSM8K, and (c) BoolQ.

Overall Performance. The results show that AttnRes-PEFT is a viable PEFT method, but in our setting it does not outperform LoRA on any of the three tasks.

On **BoolQ**, AttnRes-PEFT achieves 74.89% accuracy, slightly improving over the base model (74.28%) while remaining below LoRA (76.88%). This suggests that depth-wise aggregation contributes useful contextual refinement even for relatively shallow reasoning tasks.

On **GSM8K**, AttnRes-PEFT substantially improves over the base model (19.11% vs. 1.59%), demonstrating that the method enables non-trivial multi-step reasoning capabilities. While still slightly below LoRA (20.62%), the gap is modest relative to the large absolute gain over the frozen baseline.

On **RTE**, AttnRes-PEFT achieves 63.18%, comparable to the base model (63.54%) but below LoRA (66.79%). This indicates that for low-resource classification-style tasks, weight-space adaptation remains advantageous.

Key Observations.

- **Strongest gain on GSM8K:** AttnRes-PEFT improves from 1.59% to 19.11% on GSM8K, showing that depth-wise routing can support reasoning-heavy tasks.
- **Mixed effect on classification tasks:** On BoolQ, AttnRes-PEFT is slightly above the base model, while on RTE it is slightly below the base model.
- **LoRA remains the strongest baseline:** LoRA achieves the best accuracy on all three tasks, suggesting that weight-space adaptation is still more effective in this experimental setting.

5.2 Training Dynamics

Figure 1 shows the training loss comparison between LoRA and AttnRes-PEFT across the three benchmarks.

Stable Initialization. Due to zero-gate initialization, training starts from the frozen base-model behavior and then gradually learns useful residual routing patterns.

Smooth Convergence. Across all three tasks, the training loss decreases steadily, which suggests that the adapter can be optimized without destabilizing the frozen backbone.

Task Complexity Effects. GSM8K converges more slowly than BoolQ and RTE, which is consistent with the greater difficulty of mathematical reasoning compared with short-form classification.

5.3 Ablation Studies

Due to limited compute, we do not run a full ablation on all three tasks. Instead, we use **RTE** as a smaller benchmark for studying the effect of the **lookback window size** W.

Table 4: Effect of lookback window size W on RTE accuracy.

Window Size W	**Accuracy (%)**
1	62.45
2	63.90
4	**63.90**
8	63.54
16	63.54
20	63.54
28	63.54

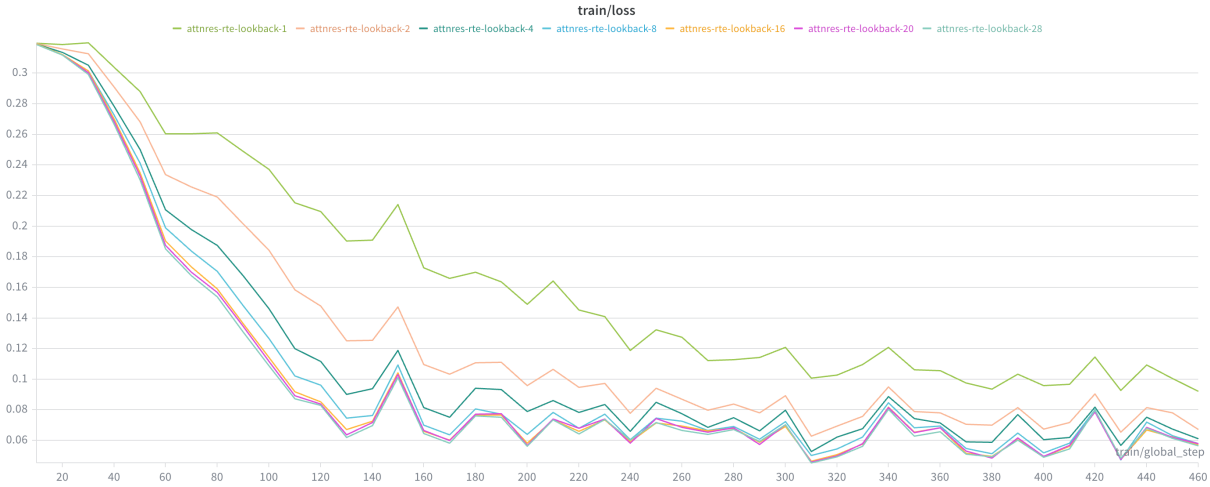
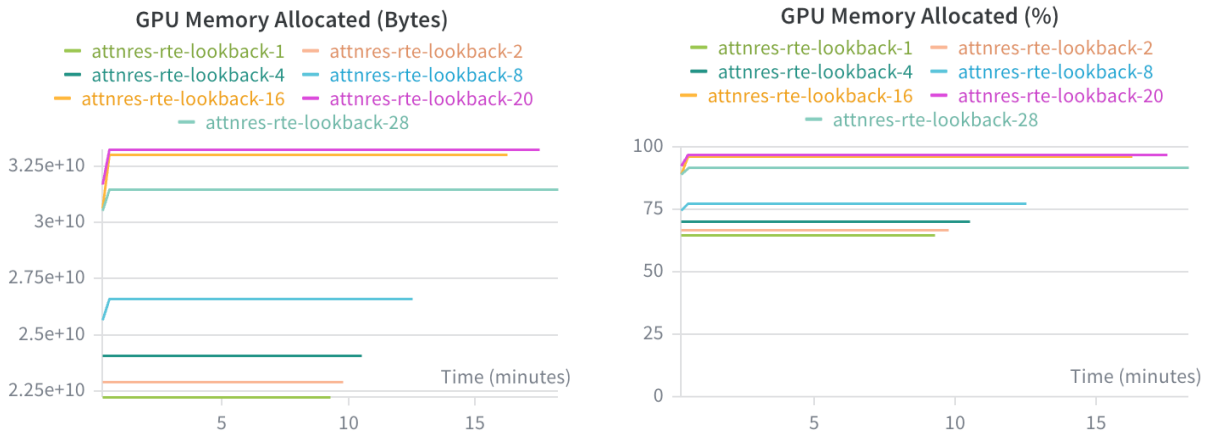


Figure 2: Ablation study on the effect of lookback window size on RTE.



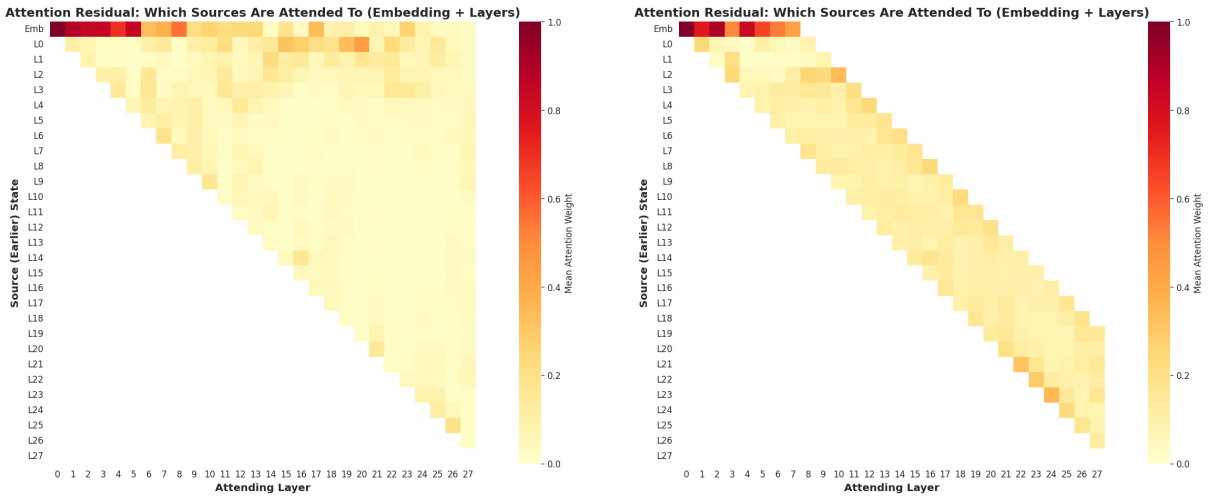
(a) Memory usage (bytes)

(b) Memory usage (% of baseline)

Figure 3: Memory ablation study for AttnRes-PEFT. (a) Absolute memory consumption in bytes across different lookback window sizes. (b) Relative memory overhead as a percentage of the frozen backbone baseline. Results indicate that restricting the lookback window (W) reduces memory footprint while preserving adaptation capacity.

5.4 Attention Pattern Analysis

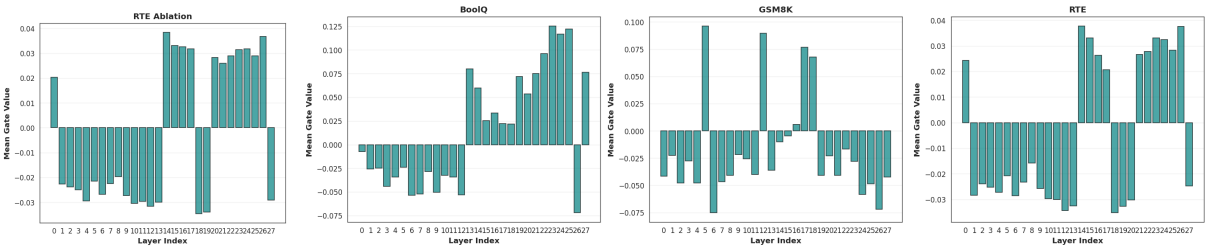
We analyze AttnRes-PEFT in two settings. **Setting A** is an RTE ablation model with a near-full-depth lookback window ($W = 28$), used to inspect routing behavior across almost the full model depth. **Setting B** is the standard main-experiment setting on BoolQ, GSM8K, and RTE, used to verify whether the same behavior holds under regular multi-task evaluation.



(a) Setting A (RTE ablation, $W = 28$).

(b) Setting B (standard setting, aggregated over BoolQ/GSM8K/RTE).

Figure 4: Depth-attention heatmaps. Each cell is the mean attention weight from an attending layer (x-axis) to a source state (y-axis), averaged over samples and token positions.



(a) Setting A (ablation).

(b) Setting B (BoolQ/GSM8K/RTE).

Figure 5: Layer-wise mean gate values.

Depth-attention patterns. Figures 4a and 4b show consistent non-uniform depth routing. In Setting A, early and middle adapter layers place substantial mass on the embedding state ("Emb"), while deeper layers shift attention toward a smaller set of intermediate/recent source layers. In Setting B, the aggregated cross-task heatmap is also clearly non-uniform, confirming that AttnRes-PEFT does not uniformly reuse previous layers. Together, these heatmaps indicate structured and hierarchical depth-wise routing rather than naive averaging.

Gate dynamics. Figures 5 and 6 present the same gate signal from complementary views. In the layer-wise mean bars (Figure 5), shallow layers are mostly near zero or slightly negative, while deeper layers are mostly positive with several layer-specific drops. The activation curves (Figure 6) confirm the same depth-wise transition and also show larger upper-layer variability in the standard multi-task setting. Overall, gate usage is depth-aware and task-dependent.

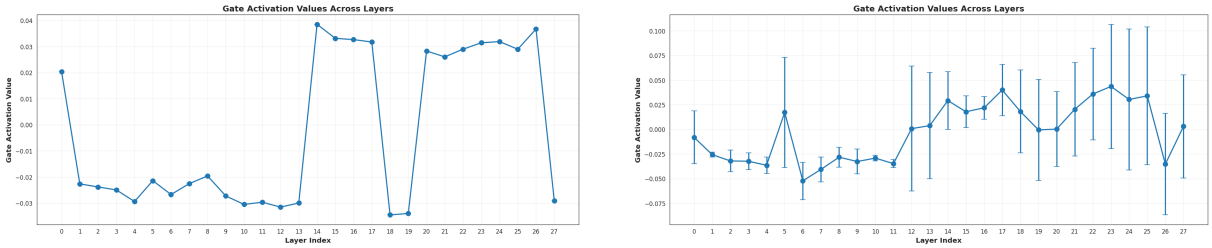
Taken together, these six plots support a consistent conclusion: AttnRes-PEFT learns hierarchical and task-dependent depth routing, remaining relatively conservative in shallow layers while increasingly activating depth-wise residual aggregation in deeper layers.

6 Discussion and Future Work

Our results show that AttnRes-PEFT is a meaningful but currently limited PEFT direction. It provides the clearest benefit on GSM8K, where cross-depth reuse appears useful for reasoning. On smaller classification tasks such as RTE, however, representation-space routing alone is less effective than weight-space adaptation, and LoRA remains stronger.

From an engineering perspective, LoRA is also much more mature. Its training, checkpointing, and deployment workflows are already well supported by existing tools, while AttnRes-PEFT still requires more custom implementation (adapter integration, save/load handling, and analysis code). In addition, AttnRes-PEFT does not yet fit into mature inference acceleration paths (e.g., standard vLLM workflows) as easily as LoRA.

Another practical limitation is scaling behavior. Increasing lookback does not increase trainable parameter count, but it does increase runtime and memory usage due to larger cross-depth aggregation. This makes the main trade-off a compute/memory trade-off rather than a parameter-budget trade-off.



(a) Setting A (RTE ablation).

(b) Setting B (mean ± std over BoolQ/GSM8K/RTE).

Figure 6: Gate activation values across layers.

Despite these limitations, the method is still valuable because it introduces a different adaptation axis for PEFT: changing residual aggregation across depth without modifying backbone weights. This direction is naturally complementary to LoRA. A hybrid design that combines weight-space adaptation (LoRA) with depth-wise routing (AttnRes) is a promising future direction.

Finally, due to compute constraints, we ran full-lookback analysis only on RTE. Extending the same analysis to BoolQ and GSM8K, and training a more general AttnRes model to study cross-task routing shifts, are important next steps.

7 Conclusion

In this work, we introduced AttnRes-PEFT, a parameter-efficient fine-tuning method that repurposes the Attention Residuals mechanism for downstream adaptation. Instead of modifying backbone weight matrices, the approach attaches a lightweight depth-wise adapter at each Transformer layer. The adapter computes a gated, attention-weighted residual correction from preceding hidden states, thereby adapting how representations are aggregated across depth while leaving all pre-trained weights frozen. We show that zero-initialization of the scalar gate makes the adapter contribution exactly zero at the start of fine-tuning, which supports stable training.

Experiments on BoolQ, GSM8K, and RTE show that AttnRes-PEFT substantially outperforms the frozen baseline—most notably on the reasoning-heavy GSM8K benchmark (1.59 % → 19.11 %)—and remains competitive on BoolQ. Although LoRA delivers stronger absolute performance across the three tasks in our setting, AttnRes-PEFT occupies a distinct region of the PEFT design space: it is extremely lightweight (only $3d + 1$ trainable parameters per layer), offers built-in interpretability through explicit depth-attention weights, and exhibits stable training dynamics thanks to its zero-correction initialization.

Ablation studies further reveal that performance saturates once the adapter can attend to a small number of preceding layers ($W \geq 2$), confirming that useful intermediate representations are concentrated locally. Layer-usage analysis demonstrates task-dependent routing patterns: classification tasks favor sharply local reuse in upper layers, while reasoning tasks distribute attention more broadly across mid-to-late layers. These findings highlight the complementary nature of depth-wise aggregation versus conventional weight-space adaptation.

AttnRes-PEFT therefore provides a new, parameter-efficient and interpretable axis for PEFT, rather than a memory-saving alternative to LoRA in its current form. Future work includes hybrid architectures that combine depth routing with low-rank weight updates, scaling to larger models and multilingual settings, and exploring its utility beyond text. By showing that architectural innovations from pre-training can be effectively transferred to the fine-tuning regime, this work opens a promising direction for more expressive yet efficient adaptation of large language models.

References

[1] Christopher Clark et al. "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions". In: *Proceedings of NAACL-HLT*. 2019, pp. 2924–2936.

[2] Karl Cobbe et al. "Training Verifiers to Solve Math Word Problems". In: *arXiv preprint arXiv:2110.14168* (2021).

[3] Neil Houlsby et al. *Parameter-Efficient Transfer Learning for NLP*. 2019. arXiv: 1902.00751 [cs.LG]. URL: https://arxiv.org/abs/1902.00751.

[4] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2021).

[5] Xiang Lisa Li and Percy Liang. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. 2021. arXiv: 2101.00190 [cs.CL]. URL: https://arxiv.org/abs/2101.00190.

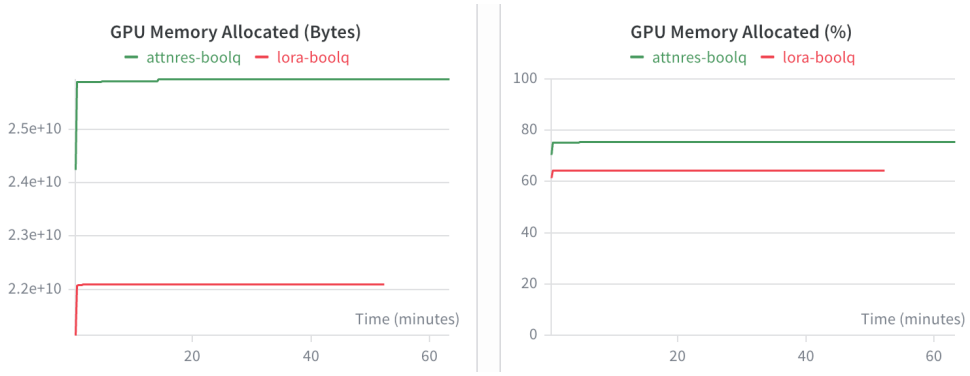
[6] Kimi Team et al. *Attention Residuals*. 2026. arXiv: 2603.15031 [cs.CL]. URL: https://arxiv.org/abs/2603.15031.

[7] Alex Wang et al. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding". In: *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*. 2018, pp. 353–355.

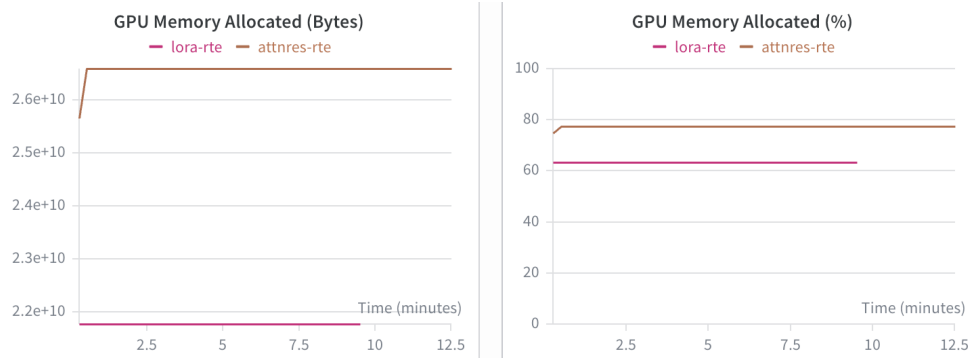
Supplementary Sheet

Table 5: Top attention sources for each layer across three datasets. Values show attention weights (only sources with weight ≥ 0.03 shown).

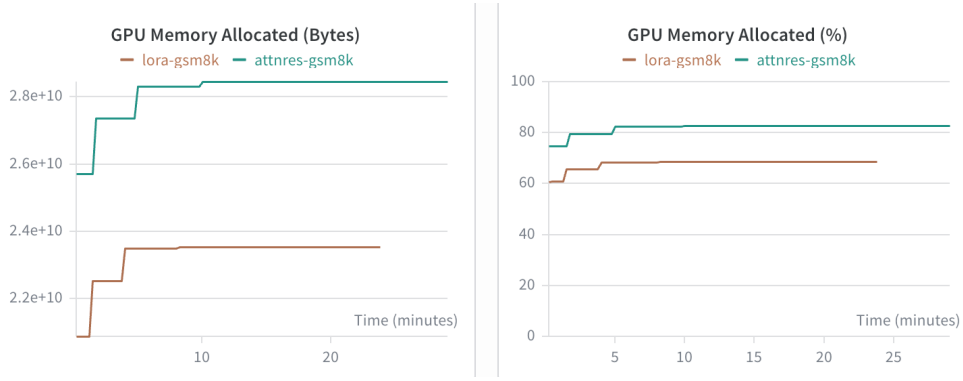
Layer	BoolQ	GSM8K	RTE
0	Emb(1.000)	Emb(1.000)	Emb(1.000)
1	Emb(0.967), L0(0.033)	L0(0.515), Emb(0.486)	Emb(0.823), L0(0.177)
2	Emb(0.976), L1(0.016), L0(0.009)	Emb(0.876), L0(0.103), L1(0.020)	Emb(0.871), L0(0.086), L1(0.043)
3	L2(0.490), Emb(0.488), L0(0.012)	L1(0.594), Emb(0.231), L2(0.147)	Emb(0.802), L2(0.096), L0(0.066)
4	Emb(0.826), L2(0.091), L3(0.045)	Emb(0.886), L3(0.058), L0(0.022)	Emb(0.802), L3(0.112), L2(0.068)
5	Emb(0.928), L3(0.033), L0(0.020)	L0(0.269), Emb(0.214), L3(0.194)	Emb(0.817), L4(0.083), L0(0.031)
6	Emb(0.501), L3(0.236), L4(0.151)	Emb(0.641), L5(0.119), L4(0.116)	Emb(0.450), L0(0.172), L5(0.119)
7	Emb(0.442), L6(0.165), L3(0.150)	Emb(0.359), L2(0.165), L4(0.118)	Emb(0.487), L2(0.154), L3(0.098)



(a) BoolQ



(b) RTE



(c) GSM8K

Figure 7: GPU memory consumption during fine-tuning with AttnRes-PEFT across three benchmarks. Each subplot shows peak memory usage (in GB) over training steps for (a) BoolQ, (b) RTE, and (c) GSM8K. Memory profiles reflect the impact of sequence length and task complexity on adapter overhead.